

Skillcase Intern Assessment

Time Limit: 36 Hours

Objective: Build a mini short-video learning platform using the required tech stack with clean architecture, smooth UI, and production-level structure.

Provided Resources

Three short-form videos are provided via Google Drive at the link below:

[Google Drive](#)

Requirements:

- Download the 3 provided videos and store them locally in the backend (e.g., /uploads folder).
- Do NOT upload video files to GitHub. Use .gitignore to exclude them.
- Videos must be served using Express static middleware.

Backend Requirements (Node.js + Express + PostgreSQL)

1. Authentication (JWT Based)

Implement the following endpoints:

POST /auth/register

POST /auth/login

GET /auth/me (protected)

Requirements:

- Password must be hashed using bcrypt.
- JWT must be issued upon login.
- JWT must be verified using middleware.
- Proper error handling and status codes must be used.

2. Shorts (Video) Module

Create a video table with the following fields:

- id (UUID or serial primary key)
- title
- description
- category
- file_path
- like_count (default 0)
- created_at

Endpoints:

POST /videos (protected)

GET /videos (return all shorts ordered by created_at DESC)

GET /videos/:id

3. Like, Comment, Bookmark System

Create required relational tables:

- likes (user_id, video_id) with composite primary key
- comments (id, user_id, video_id, content, created_at)
- bookmarks (user_id, video_id) with composite primary key

Endpoints:

POST /videos/:id/like

POST /videos/:id/comment

POST /videos/:id/bookmark

GET /videos/:id/comments

Requirements:

- Prevent duplicate likes and bookmarks.
- Increment like_count atomically in the database.
- Use transactions where necessary.

4. Middleware & Error Handling

- authMiddleware for JWT verification.
- Centralized error handling middleware.
- Proper input validation.

5. Database Rules

- Use PostgreSQL (Neon or Supabase).
- Use proper primary keys and foreign keys.
- Add index on category column.
- Use transactions where appropriate.

Frontend Requirements (React + Vite + Redux Toolkit)

Build a vertical short-video feed similar to Instagram Reels or YouTube Shorts.

Functional Requirements:

- Full-screen vertical video layout (one video visible at a time).
- Scroll vertically to navigate between videos.
- Each video must autoplay when in view.
- Like, Comment, and Bookmark buttons must be displayed as an overlay on the video.
- Comment section must open as a slide-up bottom sheet.
- Implement smooth transitions between videos.
- UI must be minimal, clean, and modern.
- Subtle animations are required (Framer Motion or GSAP may be used). Animations must enhance UX and not affect performance.

Frontend Technical Requirements:

- Use React Router.
- Use Redux Toolkit for authentication state.
- Configure Axios using a centralized API module.
- Show loading and error states appropriately.

- Implement optimistic UI update for Like functionality.

Project Structure Requirements (Mandatory)

Backend Structure:

src/

└─ routes/

└─ controllers/

└─ services/

└─ middlewares/

└─ config/

└─ utils/

Frontend Structure:

src/

└─ pages/

└─ components/

└─ redux/

└─ api/

└─ hooks/

Rules:

- No business logic inside routes.
- Use environment variables.
- Maintain clean naming conventions and readable code.

Submission Requirements

- GitHub repository (frontend + backend).
- SQL schema file (you can include this in model folder if you want).

- README with setup steps and architecture explanation.
- Clear instructions to run both backend and frontend.
- Video files must NOT be pushed to GitHub.